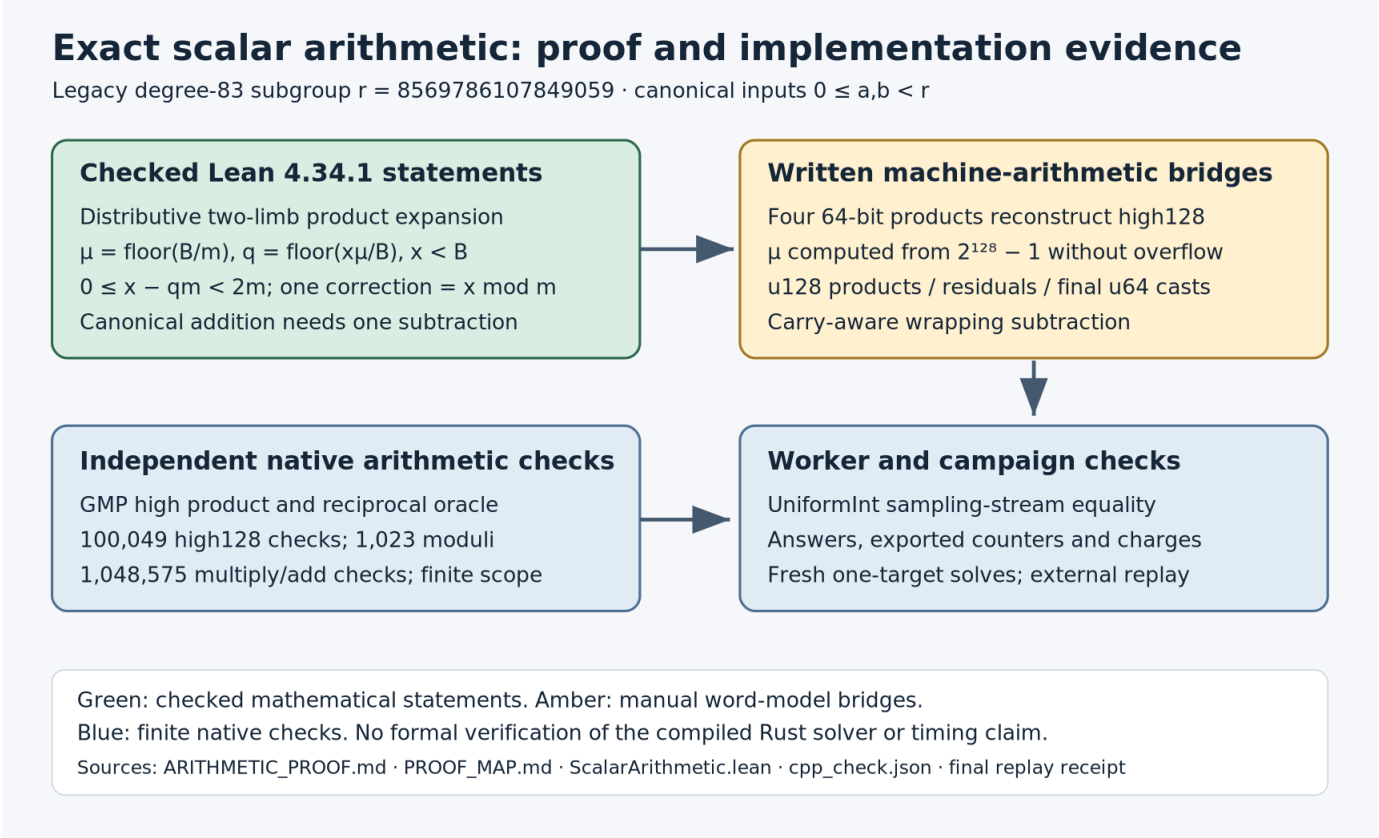
Independent degree-83 scalar audit

2026-10-12 · Exact arithmetic, formal mathematical statements and fair measurement protocol

The native campaign retained the reference scalar kernel and selected 256 lanes with DP6. Correctness of the Barrett candidate is separate from its measured performance; it did not improve arithmetic-stage process cost.



Sources and raw receipts are linked throughout. The four Lean declarations are checked mathematical statements; the compiled Rust program and word-operation bridges have the explicitly stated separate verification scope.

Exact scalar arithmetic and independent audit

The candidate changes the scalar representation inside the configured worker only. The production `StrongRhoG` implementation and the earlier frozen worker remain unchanged. Both arms use the same `Gf2_128` field, exact legacy curve, generator, signed-Frobenius quotient, jump-seed derivation and walk code.

The exact fixture is `icv1-f2m83-t6151469093347-9eba6cd5`: field `z^83+z^7+z^4+z^2+1`, curve `y^2+xy=x^3+x^2+1`, subgroup order `r=8569786107849059`, cofactor `1128547018`, and generator `(661424465139818486029201 5,886826041002924939153 9529)`. The scalar changes are constant-factor engineering; they introduce no new ECDLP exponent or curve transfer.

Proposition 1: high-product reconstruction

Put `W=2^64`, `B=W^2=2^128`, and decompose two unsigned 128-bit words as `a=a0+W*a1`, `b=b0+W*b1`, with all four limbs in `[0,W)`. Distributivity gives

`a*b=p00+W*(p01+p10)+B*p11`,

where `pij=ai*bj`. Writing each `pij=W*hiij+loij`, with its low limb in `[0,W)`, the upper 128 bits are

`p11+hi01+hi10+floor((hi00+lo01+lo10)/W)`.

This is the worker's `mul_hi_128`. The middle accumulator is below `3W` and therefore fits in 128 bits. Each partial sum of the final nonnegative terms is at most the final high word, which is below `B`; no intermediate addition overflows. The exact floor/carry reconstruction and Rust casts are supported here by this written proof and independent native tests. They are not yet a formal proof of the compiled Rust function.

The distributive polynomial expansion is checked in Lean as `N83ScalarArithmetic.limb_product_expansion`. Certificate `IDC1hc23ae1d80fc5fefa` also replays that polynomial over `F_(2^61-1)` at 64 statement-bound points and 64 additional checker-seeded points. Its `proof` field is null. That evaluated certificate does not certify integer overflow, the carry/floor formula, or the Barrett algorithm; the readable integer argument above has the larger scope.

Proposition 2: the reciprocal is exact and representable

For `2<=m<=2^64-1`, the worker stores `mu=floor(B/m)`. It computes this without representing `B` in a 128-bit word:

`mu=floor((B-1)/m)+[ (B-1) mod m = m-1 ]`.

Indeed, if `B-1=u*m+v` with `0<=v<m`, then `B=u*m+(v+1)`. The quotient increases exactly when `v=m-1`. Since `m>=2`, `mu<=B/2`, so the extra one cannot overflow the 128-bit representation. This specific representability and successor-division bridge is written and tested; its formalization status is listed separately in the proof map.

Proposition 3: one correction computes the exact remainder

Let `m>0`, `B>0`, `0<=x<B`, `mu=floor(B/m)` and `q=floor(x*mu/B)`. Define remainders `s=B-m*mu` and `t=x*mu-B*q`. Then `0<=s<m`, `0<=t<B`, and

`B*x=B*q*m+m*t+s*x`.

Since `mu*m<=B` and `q*B<=x*mu`, multiplication and cancellation by positive `B` give `q*m<=x`. Put `u=x-q*m`. The identity above becomes

`B*u=m*t+s*x`.

The first term is below `m*B`. The second term is at most `m*x`, which is below `m*B` because `x<B` and `m>0`. Thus `0<=u<2m`. Subtracting `m` exactly when `u>=m` produces a value in `[0,m)` congruent to `x`, hence exactly `x mod m`.

Lean checks this full general natural-number statement, with actual floor division definitions rather than an assumed quotient-error conclusion, as `barrett_residual_bounds` and `barrett_one_correction`. For worker inputs `a,b<m<=2^64-1`, `x=a*b<B`. Also `q*m<=x<B`, the residual is below `2m<2^65`, and the corrected result is below `m`. Consequently its multiplication, subtraction and final 64-bit cast are exact under the

canonical-input invariant. The final bridge from this natural-number theorem to the Rust word operations is manual plus native verification.

Proposition 4: canonical addition and overflow-aware subtraction

For `a,b<m`, the integer sum `S=a+b` lies in `[0,2m)`. One subtraction when `S>=m` therefore returns `S mod m`. Lean checks this as `canonical_add_one_subtraction`.

For 64-bit words, put `W=2^64` and assume `m<W`. If `S<W`, the machine sum equals `S` and the conditional test selects the same branch. If `S>=W`, the overflowing sum is `S-W`; `S>=m` and `0<=S-m<m<W`. Wrapping subtraction of `m` from `S-W` therefore produces exactly `S-m`. This establishes the worker's `(carry || sum>=m)` implementation under the invariant, including moduli above `2^63`. The word-model bridge is manual plus native tests, not a BitVec proof of the Rust operation.

Canonical-input and randomness audit

All `StrongRhoG` hot-path scalar inputs are canonical: samples are in `[0,m)` or `[1,m)`, lambda powers and collision values use exact modular arithmetic, and each canonicalization or state update maps canonical residues to canonical residues. `mul_mod`, `sub_mod` and `inverse_mod` delegate to the exact production `u64` code. The optimized `mul_in` and `add_mod` are intended for those canonical inputs; no unrestricted input equivalence is claimed.

`ExactUniform` delegates constructed sampling and both one-shot sampling methods to `UniformInt<u64>`, preserving rejection draws and RNG positions. The worker's tests compare exclusive/inclusive and constructed/one-shot streams across four ranges, including ranges above `2^63`, and compare the next RNG word after each cycle. The exact `12345*G` solve checks answers, jumps, selected canonical states, exported counters, charges, and the post-solve RNG position. No full per-step trace is exported.

Independent native checks

`cpp_check.json` records 100,049 exact high-product checks and 1,048,575 multiply/add checks across 1,023 moduli, including 2, subgroup-order neighbors, `2^50` neighbors, `2^63` neighbors and `2^64-1`. The high-product and reciprocal oracles use GMP arbitrary precision; remainders use independent exact unsigned-128 division. 41,936 tested products require the correction branch. This finite test suite supports implementation correctness; it is not a universal theorem or an independent replay of solver records.

The first C++ build failed because Boost headers were absent. That failed build is retained, and the implementation uses the installed GMP backend. All source, commands, receipts and formal verification status are retained beside this report. The final benchmark report links the exact campaign replay receipt separately.

Formal proof map

The project uses Lean `leanprover/lean4:v4.34.1`, compiler commit `5045d0056413266e57c625dcd7c365b10e377c52`, Linux x86_64. It imports only bundled `Init`; there are no external Lean dependencies. The official release archive URL and SHA-256 are in `commands.txt` and `lean_archive.sha256`. No `sorry`, `admit`, custom claim axioms, native evaluation or kernel bypasses are used.

Readable result	Lean declaration	Exact hypotheses/scope	Status
Proposition 1, distributive limb expansion	`N83ScalarArithmetic.limb_product_expansion`	All five natural-number variables; polynomial expansion before floor/carry extraction	Checked local build; axiom/fresh-kernel receipts linked below
Proposition 1, high128 reconstruction and word overflow	None	Four limbs below 2^64; exact floor/carry and Rust word operations	Written proof and independent GMP tests; machine bridge remains open
Proposition 2, representable reciprocal via `B-1`	None	`B=2^128`, `2<=m<=2^64-1`	Written proof and independent GMP tests; successor-division/word bridge remains open
Proposition 3, residual bound	`N83ScalarArithmetic.barrett_residual_bounds`	All natural `B,m,x`; `B>0,m>0,x<B`; actual `mu=B/m`, `q=x*mu/B`	Checked local build; axiom/fresh-kernel receipts linked below
Proposition 3, exact one correction	`N83ScalarArithmetic.barrett_one_correction`	Same general hypotheses; conditional subtraction equals actual `x%m`	Checked local build; axiom/fresh-kernel receipts linked below
Proposition 3, instantiated Rust arithmetic	None	Canonical residues and exact high-product/reciprocal bridge	Manual instantiation, worker tests and independent native arithmetic tests; compiled Rust equivalence remains open
Proposition 4, canonical modular addition	`N83ScalarArithmetic.canonical_add_one_subtraction`	All natural `m,a,b`; `a<m,b<m`; no assumed conclusion	Checked local build; axiom/fresh-kernel receipts linked below
Proposition 4, wrapping word implementation	None	`m<2^64`, `a,b<m`	Written two-case word proof and independent native tests; BitVec bridge remains open

The checked declarations certify the mathematical kernel statements. They do not certify the whole Rust solver, its compiled executable, timings, subgroup-order computation, target generator, or benchmark runner. Those have their own native tests, exact fixture checks, frozen provenance and independent answer replay.

`lake_build.attempt1.log`/`axioms.attempt1.log` checked an earlier version importing bundled Std. Its fresh kernel check was intentionally interrupted after about a minute to remove the unnecessary broad import. The final version imports only Init; later receipts apply to that version. Earlier syntax/algebra simplification failures remain as `lean_build.attempt1.log` and `lean_build.attempt2.log`; the direct third compile passed. Final receipt names and hashes are in `verification_receipt.json`.

The repository workflow `` `n83-scalar-arithmetic-lean.yml` `` repeats the core-only build, fresh kernel replay and four-declaration axiom audit on relevant pull requests, `main` changes and manual dispatch. It rejects incomplete/native-evaluated proof sources and unexpected transitive axioms. The local workflow syntax and shell guards are checked separately; no remote CI execution or CI outcome is claimed by this audit.

Frozen campaign audit

The pre-launch source audit passes for the stated exploratory comparison. This document reviews the protocol, not its eventual measured outcome. Final completion, interval and replay status come from the native campaign records and independent replay receipt.

Requirement	Frozen implementation	Pre-launch status
Compare with already optimized rho	Current arm is Fold/reference scalar, 64 lanes, DP6, escape off, in the same executable as the candidate	Verified by source inspection
Tune fairly	18 arithmetic runs and 81 policy runs on three separate training targets with three seeds each; select full nine-case process-wall mean	Verified by source inspection
Avoid best-seed/target selection	Complete nine-case candidate required; default/reference preferred within 2%; one policy frozen before holdout	Verified by source inspection and runner tests
Preserve failures	Every attempted row retained; unsuccessful attempt charged 60s; wrong answer rejects; no zero-cost imputation	Verified by source inspection and runner tests
Check arithmetic equivalence	All nine reference/Barrett pairs must match answers, exported outcomes, counters, preparation charges and policy	Verified by source inspection; not a full per-step trace
Check local timing noise	Five A/A pairs on the identical training target and seed, 10 launches, preceding the selection stages	Verified by source inspection
Fresh evaluation	Five new holdout targets, three fresh paired walk seeds each, 30 launches; no retuning from those results	Verified by source inspection; fixture/orbit checks have separate native receipt
One-target workload	Fresh child process and collision table for every run; no target batch or shared collision work	Verified by source inspection
Same resources	CPU3, one solver OS thread, 12 GiB address-space cap, 60s process cap; native isolation reservation and child receipts	Verified by source inspection; actual isolation receipt controls measured status
Complete cost	Primary statistic uses whole-process wall cost; online, CPU, RSS and walk counters are secondary records	Verified by source inspection
Honest interval	Per-target mean across three paired seeds, then five-target geometric ratio; 20,000 paired target-cluster bootstrap draws	Verified by source inspection and statistics tests
Complete optimization cost	Native analysis retains actual costs of all 139 children, including selection and A/A runs	Verified by source inspection
Independent correctness	Separate native C++ field/curve replay of every completed answer; sidecar not given to solver children	Required final gate; do not infer completion from producer booleans

The full budget is 139 child launches: 10 A/A, 18 arithmetic, 81 policy and 30 holdout. The selected policy is the best eligible policy in this bounded training grid after the stated 2% preference, not a global rho optimum. The primary ratio compares current and selected configurations on new targets in this campaign; it does not divide new measurements by old timings or update the earlier IC/rho ratio using mismatched IC evidence.

The five-target cluster interval reflects the target/seed law and this VM. It does not turn 15 seeded runs into 15 independent target clusters and does not include selection-stage uncertainty as an unconditional search estimator. A/A supplies an empirical local noise diagnostic, not a certificate of physical host exclusivity or an ecbench L2 admission. Nominal monitoring every 5ms adds scheduling and poll-body overhead; there is no strict 5ms wall-clock bound under arbitrary scheduling.

No calibrated operation-equivalent S, independent-host timing audit, full walk trace, asymptotic advantage or deployed-curve conclusion is claimed. The final report must state all failed and invalid attempts, actual isolation status, whole-process and online intervals, confidence limits, arithmetic-mean comparison and the independent replay digest.